# PaperProof Whitepaper

A Protocol for Durable Digital Artifacts on Sui and Walrus

PaperProof Labs
paperproof.labs@gmail.com
https://paperproof.site/
https://github.com/PaperProofLabs

Draft v0.2
June 2026

## Copyright and Use Notice



**Abstract**

Important digital works increasingly move across unstable boundaries: storage links, social posts, repositories, preprint pages, datasets, software releases, chat archives, and application databases. These surfaces are useful, but none of them alone gives the work itself a neutral, durable, verifiable, and composable protocol identity.

PaperProof is a Sui-native protocol for durable digital artifacts backed by Walrus content storage [1, 3]. It treats a work not as a single uploaded file, but as an artifact series: a continuing object with typed versions, content commitments, official comments, likes, governance-aware parameters, canonical events, and developer-facing SDKs. Recent protocol surfaces extend this model to agent-readable infrastructure: official Copilot prompts are stored as versioned PaperProof artifacts, and wallet-linked Copilot memory is discovered through a lightweight on-chain registry while private memory bodies remain in MemWal and Walrus. The goal is not to replace academic archives, software platforms, storage networks, social media, or community forums. The goal is to give important digital works a protocol layer beneath and beside those systems.

This whitepaper explains the vision, user value, ecosystem role, developer surface, governance direction, PPRF relevance, sustainability principles, roadmap, and risks of PaperProof. Detailed object structures, error codes, and event fields belong in the yellow paper; formal system analysis belongs in the academic paper. This document is the ecological argument: why PaperProof should exist, who can build on it, and how it can grow without pretending to solve every social, financial, or editorial problem at launch.

# Contents

# 1 Executive Summary

PaperProof is a protocol for durable, verifiable, and community-readable digital artifacts. A PaperProof artifact may be a preprint, blog post, technical report, dataset, software release, or generic file. The artifact's large content lives on Walrus, while its protocol identity and lifecycle live on Sui.

The central idea is simple: important works deserve more than a file link and an application page. They should have an identity that can survive frontend changes, repository migrations, social-feed decay, organizational changes, and market cycles. That identity should support versions, content verification, official interaction objects, events for indexers, and enough structure for applications and agents to interpret the work safely.

PaperProof begins with a practical implementation rather than only a thesis. It has mainnet contracts, a deployment manifest, an official static application, TypeScript, Python, and Rust SDKs, Walrus content helpers, governance flows, query and watch surfaces, and documentation. These pieces form a small but real protocol seed.

## 1.1 The Short Version

- PaperProof gives digital artifacts durable protocol identities.
- Sui stores compact authoritative state; Walrus stores large content.
- Artifact series support typed versions instead of one-off uploads.
- Official comments and likes are bound to artifact objects.
- PPRF is used for governance and protocol participation signals.
- SDKs make PaperProof usable from frontend apps, scripts, notebooks, and indexers.
- PaperProof is open to third-party apps; the official website is only one entrance.
- Governance, fees, incentives, and community operations are intended to grow gradually.

# 2 The Problem: Digital Works Without Protocol Identity

Digital works are everywhere, but their identities are fragmented. A paper may be on a preprint server. A dataset may be in one storage system. A software release may be on a repository host. A protocol article may be in a blog. A discussion may happen on social media. Years later, the links, accounts, websites, and application databases around the work may have changed.

This creates a gap between the content and its public life. A file hash can identify bytes, but it does not describe the artifact's type, owner, version lineage, official comments, likes, governance history, or canonical event stream. A web page can provide context, but it usually belongs to one application operator. A repository release can be useful, but it is not a neutral protocol identity. A social post can create attention, but it is not durable infrastructure.

PaperProof addresses this missing layer. It asks: what if a digital work could have a durable identity independent of one frontend, while still remaining easy to use from ordinary applications?

## 2.1 Why This Matters

Durable artifact identity matters because modern knowledge is not static. A preprint may gain new versions. A dataset may be reused by another project. A software release may need verifiable package references. A technical report may become governance evidence. A blog post may become part of a community's memory. A forum thread may host long-term discussion around a protocol artifact.

Without protocol identity, every application rebuilds its own partial truth. One system knows where the file is. Another knows the comments. Another knows the versions. Another knows the author. Another knows the community reaction. The artifact itself remains scattered.

# 3 The PaperProof Vision

PaperProof's vision is to become a durable artifact identity layer for Sui and Walrus. It should be small enough to survive, structured enough to integrate, and open enough for communities to build multiple applications on top of it.

The concise value proposition is: PaperProof turns Walrus blobs into verifiable, versioned, discussable, and agent-readable knowledge artifacts. Walrus makes content durable. PaperProof adds the protocol coordinate system that lets applications ask what the artifact is, which version is current, which interactions are official, and what structured context an agent can safely consume.

The first official application is not meant to be the only PaperProof application. It is a reference interface: Explore, Publish, Governance, My Space, Docs, Blog, Forum, and PaperProof Copilot. Other people should be able to build their own portals, dashboards, indexers, notebooks, mobile apps, institutional archives, community forums, and agentic workflows using the same protocol facts.

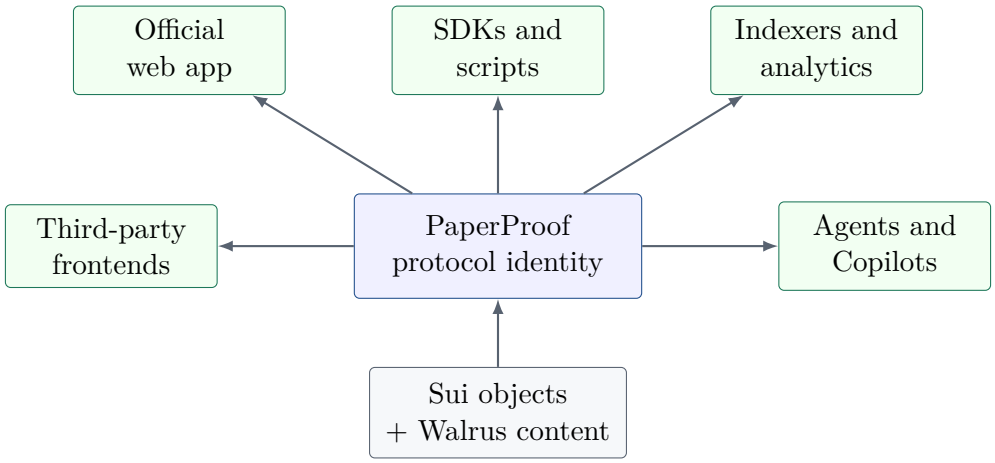


Figure 1: PaperProof is intended as a protocol layer, not a single website.

## 3.1 What PaperProof Should Become

PaperProof should become:

- a durable coordinate system for digital artifacts;
- a publishing and versioning layer for works that need public commitments;
- a content gateway that makes Walrus easier to use in artifact workflows;
- an event surface for indexers, analytics, and community applications;
- a developer platform with SDKs in multiple languages;
- a governance-aware substrate for protocol evolution;
- a foundation for agent-readable artifact workflows.

### 3.2 What PaperProof Should Not Become

PaperProof should not pretend to be everything. It should not become a monolithic social network, a full academic review authority, a universal storage network, a centralized repository host, a legal compliance oracle, or a guaranteed reward machine. Its core job is to record durable facts around artifacts. Communities, applications, reviewers, developers, and markets can interpret those facts.

## 4 Ecosystem Position

PaperProof is easiest to misunderstand when it is introduced as only a blog, a forum, a preprint server, or a release page. Those are application surfaces. The protocol is broader: it is an artifact layer that sits between the base infrastructure of Sui and Walrus and the many products that can be built above them.

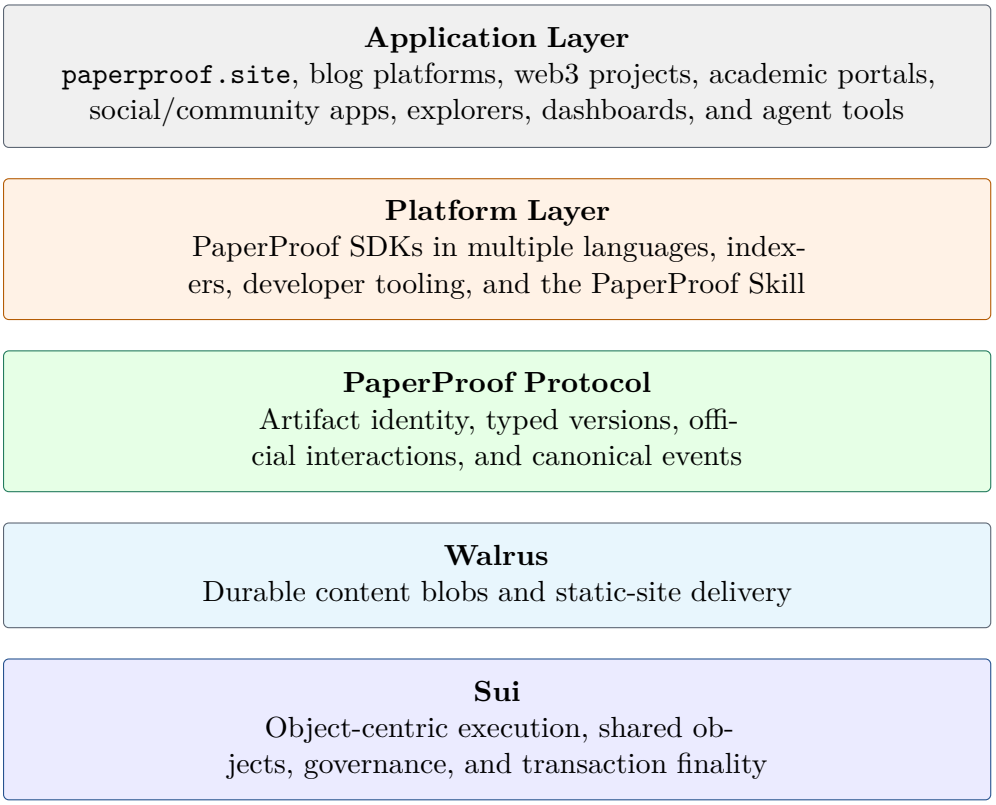


Figure 2: PaperProof's ecosystem position in the Sui and Walrus stack.

The strategic point of this stack is that PaperProof does not need to win as a single website in order to matter. If the protocol becomes the accepted coordinate system for high-value artifacts, then many different clients, communities, institutions, and products can reuse the same canonical series and version history.

### 4.1 Web3 Counterparts

PaperProof does not have one exact one-to-one competitor in web3. The closest overlap comes from three adjacent families:

- onchain publishing systems;
- open social and creator protocols;

- ownership-oriented content infrastructure.

Projects in those families generally optimize around profiles, posts, feeds, subscriptions, creator distribution, or broad social coordination. PaperProof optimizes around the artifact series: the continuing work, its exact versions, its verifiable content references, and the official interaction objects bound to it.

This distinction matters. A protocol centered on feeds competes mainly for attention and distribution. A protocol centered on artifacts can become shared infrastructure for technical papers, software releases, datasets, official guides, governed records, and agent-readable outputs that many frontends may want to render differently.

Table 1: Representative web3 counterpart families.

| **Family** | **PaperProof difference** |
|---|---|
| Onchain publishing products such as Mirror or Paragraph | Strong at author-facing publishing and distribution, but primarily product-shaped rather than artifact-protocol-shaped. |
| Open social protocols such as Lens or Farcaster | Strong at profiles, graphs, channels, and distribution, but not centered on typed artifact lineage and official version history. |
| Ownership-oriented content systems such as Crossbell or DeSo | Closer to protocolized content ownership, but still more social/account-centric than artifact-series-centric. |

This means PaperProof can coexist with, integrate beneath, or be embedded by other web3 products rather than competing only as a destination website. A Mirror-like writing surface, a research portal, or a social app could all use PaperProof as the durable artifact layer while keeping their own audience and distribution logic.

### 4.2 Web2 Counterparts

There is no single web2 product that fully matches PaperProof. The closest functional comparison is a stack:

Ghost or Substack + arXiv or OSF + Zenodo + GitHub Releases + Discourse

That comparison is useful because it shows that PaperProof is not trying to replace just one familiar surface. It unifies several workflows that are usually split across different platforms:

Table 2: Web2 functional counterparts to PaperProof.

| **Functional family** | **PaperProof relationship** |
|---|---|
| Long-form publishing | Essays, official notes, and durable docs can live as versioned artifacts rather than only as CMS pages. |
| Research and preprints | Manuscripts, reports, and supporting records can share one typed artifact model rather than separate repository logic. |
| Data and archival release | Datasets and reproducibility packages can inherit artifact identity, version history, and governed visibility. |
| Software release records | Source archives, package hashes, changelogs, and release lineage become first-class protocol records. |
| Structured discussion | Artifact-bound comments and likes give each series an official interaction surface without requiring one closed forum database. |

Web2 platforms usually rely on the application database as the final authority. PaperProof shifts authority toward protocol state plus verifiable content commitments, so a durable work can remain reconstructable even when one website changes policy or disappears.

### 4.3 Structural Advantages

PaperProof's strongest advantages are structural rather than cosmetic.

- **Artifact-centric rather than feed-centric.** The primary object is the continuing work, not the post stream.
- **Version history is protocol core.** Interfaces, SDKs, indexers, and agents can reason about exact lineage instead of treating history as an afterthought.
- **Multiple artifact families share one model.** Preprints, reports, datasets, software releases, long-form notes, and generic files can all use one identity and version framework.
- **Control can be assetized without rewriting history.** Control rights can move through a dedicated controller asset while prior versions, authorship, and license commitments remain intact.
- **Agent-readable by design.** Typed state, prompt routing, and memory-adjacent registries make the protocol easier for AI systems to consume safely.
- **Compatible with many frontends.** The official site is one interface, not the only valid interpretation surface.

These advantages do not guarantee distribution. Social-first products may still be stronger at viral growth or audience tools. But where exact versions, official bindings, evidence, and long-term machine readability matter more than short-lived feeds, PaperProof has a deeper protocol position.

## 5 What Is a PaperProof Artifact?

A PaperProof artifact is a continuing protocol object, not merely a file. It has a series identity, one artifact type, a human-readable artifact code, a controller, one or more typed versions, content commitments, Walrus references, official comments, official likes, status fields, timestamps, and events.

### 5.1 Artifact Series

The series is the continuing identity. A software project may have many releases. A preprint may have revised versions. A dataset may be corrected. A technical report may become more complete. A blog post may receive a canonical update. A generic file may be replaced by a clearer version. PaperProof models these as versions under one series rather than unrelated uploads.

The protocol also separates content license from operational control. The right to publish the next version, manage series-level metadata, and control official discussion policy can travel with a dedicated controller asset. This makes artifact control portable without rewriting historical authorship, license terms, or prior version commitments. In effect, a PaperProof artifact can be a durable digital work and a transferable control surface at the same time.

Table 3: Initial PaperProof artifact types.

| Type | Typical use |
|---|---|
| `preprint` | Manuscripts, research drafts, academic-style papers. |
| `blog_post` | Protocol announcements, community posts, essays, official updates. |
| `technical_report` | Design notes, audits, specifications, engineering reports. |
| `dataset` | Data archives, schemas, reproducibility packages, research data. |
| `software_release` | Source archives, package hashes, changelogs, release records. |
| `generic_file` | Miscellaneous files that do not yet need a specialized type. |

### 5.2 Typed Versions

The six initial artifact types are:

These types are intentionally limited. Type scarcity helps SDKs, indexers, frontends, and governance understand what a record means. User-facing labels, communities, tags, and themes can be flexible at the application layer, but protocol-recognized types should remain stable and governable.

## 6 Why Sui and Walrus

PaperProof depends on two complementary foundations.

Sui provides object-centric protocol state. PaperProof objects map naturally to Sui's model: artifact series, type registries, typed version records, comments trees, likes books, governance vaults, fee managers, proposals, and votes. This object model helps the protocol represent relationships explicitly instead of hiding them inside one application database [1, 2].

Walrus provides storage for large content. PDFs, datasets, source archives, images, long comments, and static sites should not be stored as large on-chain objects. Walrus lets PaperProof keep content available while Sui records compact commitments, references, and state transitions [3].

Together, Sui and Walrus make PaperProof possible as a living artifact protocol: transactions can represent meaningful artifact actions, while large content remains in a storage layer designed for that role.

Neither layer is decorative. Without Sui, artifact identity, ownership, version lineage, interaction bindings, governed prompt routes, and official memory availability would fall back to application database conventions. Without Walrus, PaperProof would become either a thin link registry or a conventional hosted website with chain receipts. The protocol value comes from combining verifiable object state with durable content workflows.

### 6.1 Programmable Workflows, Not Ceremonial Registrations

PaperProof is designed for artifact actions to feel like normal application workflows. Publishing, adding a version, commenting, liking, voting, and registering agent-memory capability are composable Sui operations. Where compatible, Programmable Transaction Blocks can group related Move calls so users approve a coherent workflow rather than a sequence of low-level module steps. Walrus reserve and certify phases may still require separate stages, but the user experience should expose the meaningful artifact action.

## 7 Protocol Layers

PaperProof can be understood as five layers:

1. Content layer: Walrus blobs and static sites.

2. Protocol state layer: Sui objects and Move contracts.

3. Event layer: canonical events for applications and indexers.

4. Developer layer: SDKs, examples, query providers, watch APIs, and CLIs.

5. Application layer: official app, third-party sites, forums, dashboards, agents, and data tools.

This layering is important. The protocol does not need every future product feature in the Move contracts. It needs stable facts. Applications can build different experiences around those facts.

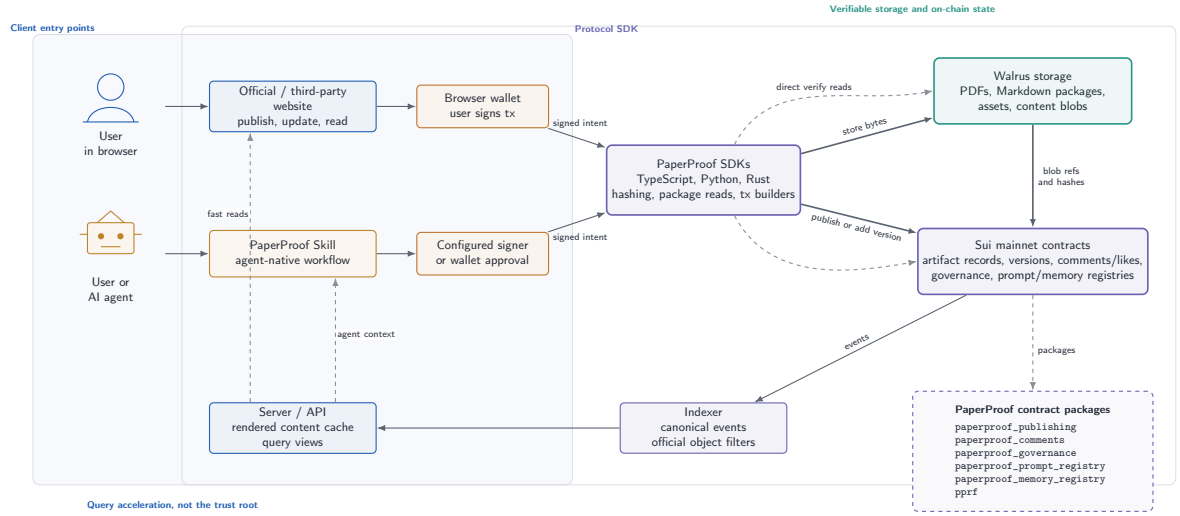


Figure 3: PaperProof application architecture and workflow: users and agents enter through official or third-party websites and the PaperProof Skill, while Sui contracts, Walrus storage, the server indexer, and SDKs coordinate one shared artifact protocol.

# 8 Official App and Open Entry Points

The official PaperProof web application is a reference interface. It helps users explore artifacts, publish new works, add versions, read details, comment, like, vote, claim locked funds, view wallet state, browse docs, read official posts, and participate in forums.

But the official app is not the protocol's only entrance. A university group could build a research portal. A software community could build a release dashboard. A data team could build an analytics indexer. A mobile developer could build a lightweight artifact browser. An AI developer could build an agent that summarizes artifact histories and governance proposals. A wallet could show PaperProof-related objects and transactions natively.

## 8.1 Official Does Not Mean Exclusive

PaperProof Labs may operate official deployments, official manifests, official docs, and an official website. That gives the ecosystem a trusted starting point. It should not prevent truthful third-party integrations. Third-party applications should be free to build on official PaperProof deployments as long as they do not misrepresent themselves as official or confuse users about authority.

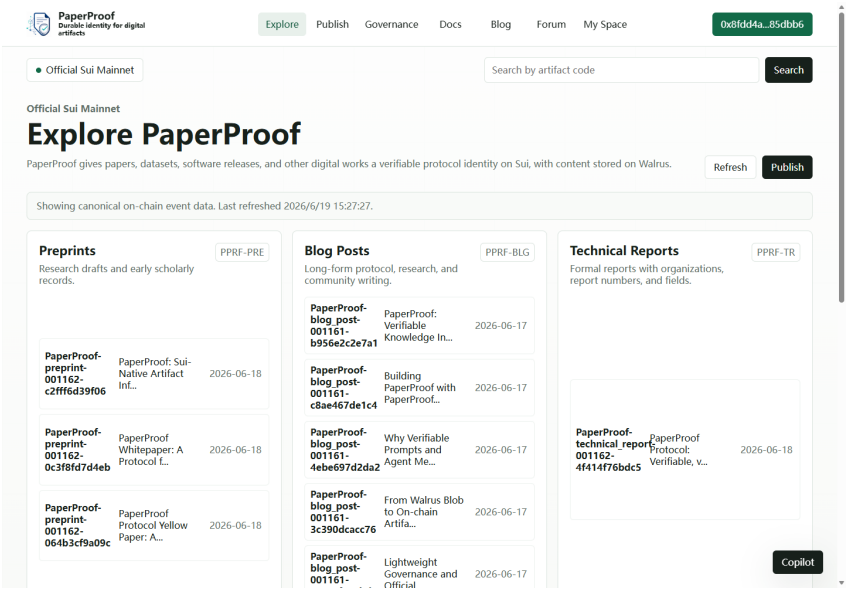


Figure 4: The official PaperProof application presents protocol artifacts through a static web interface backed by Sui state, Walrus content, and the reference indexer.

## 8.2 First-Party Dogfooding

The official website is also intended to be one of the first users of the protocol. Docs, Blog, and Forum are not merely pages about PaperProof; they can be represented by PaperProof artifacts and therefore exercise the same storage, versioning, comment, and indexing paths expected from third-party applications.

Table 4: Official app surfaces backed by PaperProof artifacts.

| **Surface** | **Protocol-backed behavior** |
|---|---|
| Docs | Published as artifact series with versioned content. The official comments tree can be locked or archived so documentation remains readable without public discussion threads. |
| Blog | Published as `blog_post` artifacts with summaries, full content, artifact codes, versions, and official comments. |
| Forum | Each topic can be represented as a `generic_file` artifact carrying a Markdown content package and an official comment tree used as the discussion thread. |
| Copilot prompts | Published as `generic_file` artifacts and selected through the prompt registry by app route. |
| Memory descriptor | Published as a prompt artifact that explains Copilot memory semantics, schema, and version policy. |

This dogfooding matters because it turns the official site into a running demonstration of the protocol's value. A visitor can see that PaperProof is not only a storage idea; it is an application substrate for versioned knowledge, discussion, and agent-readable context.

The current official app is no longer merely a pure static reader. Its shell remains a static wallet-first application, but selected public surfaces now use a server-assisted reference indexer. Explore, official Docs, Blog, and Forum can load through API views and warm caches derived from protocol state and Walrus content. This improves responsiveness without making the server the source of truth: wallet actions still require user approval, and artifact facts still resolve back

to Sui objects, Walrus references, content hashes, and deployment manifest bindings.

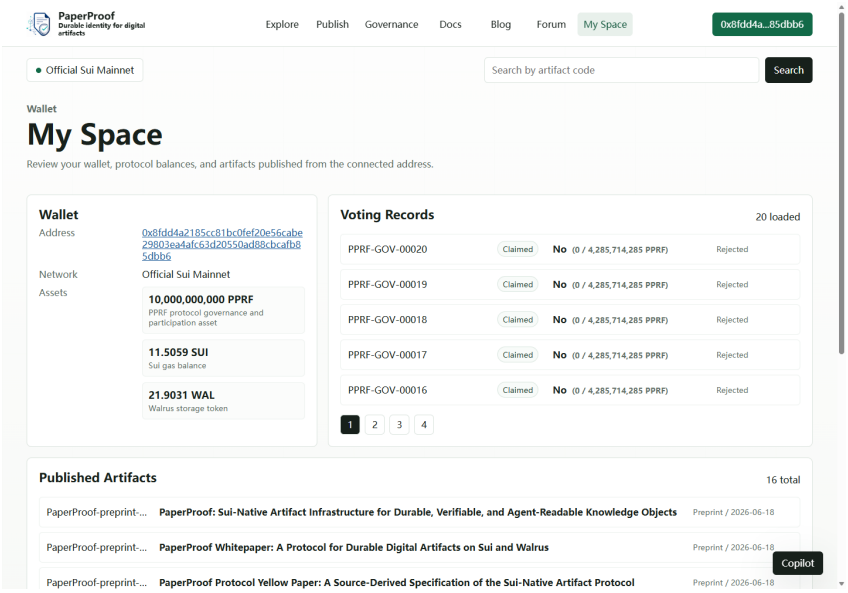


Figure 5: A connected-wallet view of My Space illustrates how the application can combine wallet-local authority with indexer-assisted public protocol views.

# 9 Application Potential

PaperProof is deliberately broader than a preprint archive. Its core primitive is a durable knowledge artifact, which can support both human-facing and agent-facing products.

Table 5: Potential application families.

| **Application family** | **PaperProof contribution** |
|---|---|
| Research and open science | Versioned manuscripts, datasets, reproducibility packages, technical reports, and durable citations. |
| Software communities | Release records, source and package commitments, changelogs, governance evidence, and discussion threads. |
| Protocol communities | Official docs, blogs, forums, proposal context, audit materials, and upgrade records represented as artifacts. |
| Agentic workflows | Durable prompts, memory descriptors, reusable reports, datasets, logs, and intermediate outputs that agents can resolve over time. |
| Artifact marketplaces and stewardship transfer | Controller assets can let a series move between individuals, teams, funds, labs, or acquiring platforms without breaking public version history. |
| Independent portals | University archives, community frontends, wallets, explorers, dashboards, and indexers using the same public protocol facts. |

These applications do not need to share one presentation layer. They share a typed protocol substrate. That distinction is central to PaperProof's long-term potential: the same artifact can outlive a particular site and remain usable by future frontends, indexers, and agents.

Because artifact control is separable from content license, the application surface can also support a broader artifact economy. A research series, a technical publication line, a curated data product, or an open-source release track can keep immutable history while its control rights

move between wallets, organizations, or market participants through ordinary Sui asset flows.

This possibility matters strategically because it gives PaperProof a second growth surface beyond publishing alone. The protocol can support not only creation and reading, but also stewardship transfer, curation businesses, treasury-owned knowledge assets, community-maintained release lines, and artifact portfolios that remain legible to outside applications.

# 10 SDK and Developer Ecosystem

PaperProof provides three SDK tracks with different personalities.

Table 6: SDK roles.

| SDK | Primary role |
|---|---|
| TypeScript | Frontend and Node.js integration, browser-wallet workflows, transaction builders, query providers, watch APIs, ContentService. |
| Python | Scripts, notebooks, batch queries, CSV/JSON exports, operational tooling, research analysis, simple automation. |
| Rust | High-performance indexers, checkpoint ingestion, persistent cursors, database sinks, backend services, enterprise-style data infrastructure. |

This split is deliberate. A browser developer, a data analyst, and an indexer operator do not need the same interface. The protocol should feel natural in each environment.

## 10.1 Developer Goals

PaperProof SDKs should make common actions short and safe:

- load the active deployment manifest;
- query recent artifacts;
- read artifact details and versions;
- build transactions without executing them;
- execute with browser wallet or keypair;
- parse typed transaction results;
- handle provider failure honestly;
- query comments, likes, proposals, and voting records;
- resolve controller state and artifact-control transfers without conflating them with content licenses or prior authorship;
- publish, read, verify, extend, and transfer Walrus content through a simplified ContentService;
- watch canonical events without accepting look-alike package events.

## 11 Indexers, Data Services, and Watchers

Long-term protocols need data infrastructure. PaperProof events should be available to frontends, explorers, dashboards, research tools, and community applications. But events must be interpreted carefully. A raw event name is not enough. Indexers must verify package IDs, registry IDs, root bindings, series bindings, comments tree IDs, likes book IDs, and governance config bindings.

The Rust SDK is designed to make this path stronger. It can support checkpoint-oriented ingestion, persistent cursors, idempotent event IDs, SQLite/Postgres sinks, backfill, tail mode, metrics, and deployment drift policies. This is the path from a working protocol to a dependable data ecosystem.

The reference indexer has also become an application acceleration layer. It can serve Explore summary, type list, search, and lookup views while preserving compatibility with older array-shaped endpoints. It can prewarm official Docs/Blog/Forum content, cache verified markdown or package output, refresh the cache after tailing relevant Sui events, and hydrate latest version objects when event payloads alone do not contain the full type-specific fields. These choices are intentionally conservative: they speed up common reads but keep canonical acceptance rules, deployment drift checks, and content verification visible to clients.

The distinction is important for ecosystem trust. A reference API may expose health, metrics, search, artifact detail, activity, governance, My Space, and analytics views, but those views are conveniences over replayable protocol facts. A third-party indexer can replace the API, search engine, cache policy, or ranking layer without replacing PaperProof itself.

## 12 PaperProof Copilot and Agentic Interfaces

PaperProof Copilot is a browser-side assistant pattern. It can read structured page data, protocol prompts, and visible state to explain what users are seeing. It can clarify artifact details, voting state, locked funds, publish forms, comment behavior, and safety checks.

The prompt layer is itself protocol-native. Official prompts are published as `generic_file` artifacts, registered by application and route, and resolved through the active prompt registry. A route may follow the latest artifact version or pin a specific version. This makes official prompts traceable, versioned, auditable, and replaceable without treating the static web bundle as the only source of truth.

Copilot memory follows a different boundary. The private memory body belongs in MemWal and Walrus, not in PaperProof Move objects. PaperProof stores only governed discovery metadata: the wallet, app, memory identifier, provider, namespace root, descriptor artifact, version policy, and availability state. The official app can therefore check whether a wallet has one active usable memory entry for the app, while the user's private preferences and summaries remain outside public chain state.

Copilot does not sign transactions. It does not hold private keys. It does not replace the wallet. This separation is central to PaperProof's Agentic Web position: AI can help users understand and prepare actions, but authorization should remain with human-approved wallets and Sui object permissions.

PaperProof artifacts are suitable for agentic workflows because they are structured. Agents can read types, versions, comments, proposals, and events instead of scraping arbitrary pages. This opens the door to artifact summaries, research assistants, governance explainers, indexer diagnostics, and content maintenance agents.

This agentic direction is already being dogfooded. The PaperProof Skill package has been published as a PaperProof Software Release artifact, demonstrating that an AI-facing protocol client can itself be represented by the protocol it helps operate. The important distinction is that agents should prefer SDKs, manifests, indexers, and direct object reads over website automation.

The official website is a human entry point; the skill layer is a protocol client for assistants.

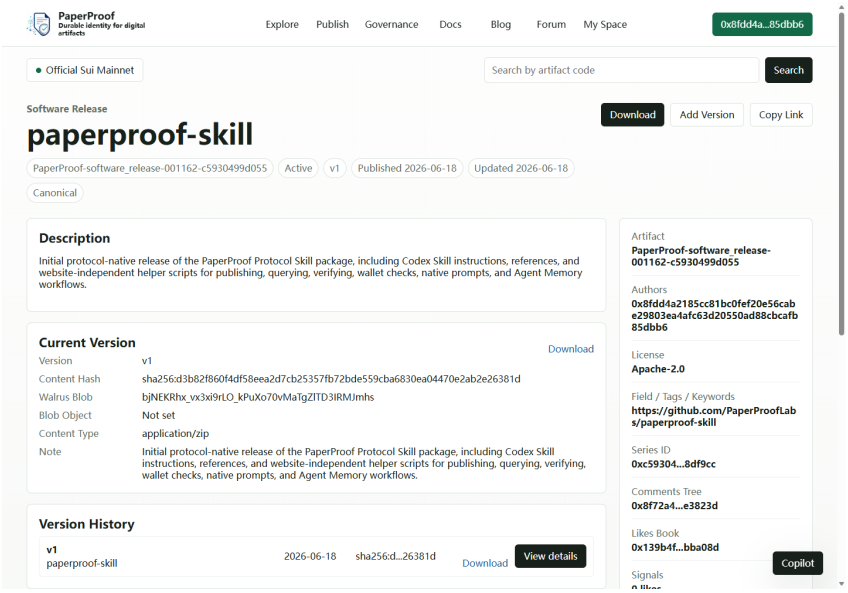


Figure 6: The PaperProof Skill release is itself a protocol artifact, showing how agent-facing tooling can be published, versioned, and verified through the same artifact model.

# 13 PPRF Utility and Governance Direction

`PPRF` is the PaperProof governance and participation asset. It currently supports governance voting and proof-of-holding participation signals such as likes. It is not a guarantee of revenue, yield, adoption, or content quality.

## 13.1 Governance Philosophy

PaperProof should not pretend to be fully decentralized on day one. Early-stage protocols need security response, deployment discipline, SDK updates, frontend repair, manifest maintenance, and practical recovery paths. The right path is progressive decentralization: begin with accountable early maintenance, then move more parameters and authority into transparent governance as the protocol, community, and tooling mature.

## 13.2 Early Governance Scope

Early governance should focus on limited, high-signal parameters:

- fee levels;
- artifact type enablement;
- governance configuration;
- action enablement;
- operator and authority transitions;
- direct authority reduction over time;
- future type activation;

- official prompt and memory capability availability.

This scope is intentionally conservative. Governance should first protect the protocol from obvious parameter abuse and centralization risk. It should not rush into complicated financial promises or vague reward mechanics.

## 14 Fees, Sustainability, and Anti-Spam

Fees in PaperProof are primarily resource and spam boundaries. They are not designed as a maximized revenue extraction system. Publishing, adding versions, and comments can carry fees configured through governance-aware mechanisms. Fees may be low or even zero in some periods, but the protocol still needs bounded metadata, object binding, status checks, and canonical filters.

Sustainability should develop gradually. In early phases, PaperProof Labs may carry much of the maintenance burden. Over time, the ecosystem may need grants, bounties, infrastructure funding, content-maintenance policies, and contributor support. These mechanisms should be transparent, modest, and auditable before they become ambitious.

## 15 Incentives and Community Growth

PaperProof should avoid promising simplistic "publish-to-earn" incentives. Artifact quality, research value, social usefulness, and software reliability cannot be produced by token rewards alone. Bad incentives can fill a protocol with spam faster than good content.

Instead, early incentives should support useful ecosystem work:

- SDK examples and integrations;
- indexers and dashboards;
- documentation and translations;
- official and community blog posts;
- artifact curation and educational collections;
- frontend experiments;
- tools for Walrus content maintenance;
- agents that help users understand protocol state safely.

The community should grow around credible artifacts and useful tools, not only around token speculation. PPRF governance can become meaningful only if there is an ecosystem worth governing.

## 16 Roadmap

The roadmap is directional, not a promise of dates, returns, or specific token incentives.

### 16.1 Phase 0: Mainnet Protocol Seed

The seed phase establishes the minimum durable protocol:

- mainnet contracts for publishing, comments, likes, governance, and fees;
- initial artifact types;
- official deployment manifest;
- TypeScript, Python, and Rust SDKs;
- official static web application;
- reference indexer APIs for Explore and official public content caches;
- protocol-native Copilot prompt registry;
- lightweight Copilot memory registry for governed discovery metadata;
- protocol-published flagship artifacts, including papers, a benchmark dataset, and the Paper-Proof Skill software release;
- yellow paper, academic paper, slides, and developer documentation;
- opt-in mainnet integration scripts and examples.

### 16.2 Phase 1: Usable Protocol Surface

This phase focuses on making PaperProof pleasant and credible for early users:

- polish Explore, Publish, Governance, My Space, Docs, Blog, and Forum;
- keep official public content caches event-driven with periodic reconciliation as a fallback;
- publish formal-verification evidence and mainnet manifest drift checks as ordinary review surfaces;
- strengthen PaperProof Copilot with native prompts and wallet-linked memory;
- improve Walrus upload/read/verify/extend/transfer flows;
- improve examples and tutorials;
- harden query providers and watch APIs;
- clarify user safety, wallet prompts, and error explanations.

### 16.3 Phase 2: Developer and Data Ecosystem

This phase expands beyond the official website:

- third-party frontends and research portals;
- high-performance indexers;
- analytics dashboards;
- notebook and CLI workflows;
- CSV/JSON export tools;
- event subscriptions and watcher services;
- grants or bounty experiments for useful ecosystem work.

### 16.4 Phase 3: Governance Expansion

Governance can grow as the protocol becomes more used:

- more parameter changes through PPRF proposals;
- clearer treasury or ecosystem-support policy if needed;
- transparent upgrade procedures;
- stronger role-transition processes;
- gradual reduction of direct authority;
- community-led proposals for new artifact types or application standards.

### 16.5 Phase 4: Durable Knowledge Network

The long-term ambition is a multi-entry artifact network:

- multiple frontends and community portals;
- multiple indexers and data services;
- academic, open-source, dataset, protocol-documentation, and community publishing use cases;
- agent-readable artifact histories;
- long-lived content and governance records on Sui and Walrus;
- PaperProof as a recognized digital artifact identity layer.

## 17 Risk, Safety, and Non-Guarantees

PaperProof makes artifact commitments more durable and verifiable. It does not make every artifact true, original, legal, useful, safe, or valuable. A bad paper can be published. A misleading dataset can be uploaded. A spam comment can be posted. A token vote can be apathetic or concentrated. A frontend can still have bugs. A storage lifecycle can still require maintenance.

### 17.1 User and Developer Risks

Users should understand wallet prompts before signing. Developers should apply canonical event filters and drift checks. Indexers should distinguish failed queries from empty state. Frontends should not report "no comments" or "no votes" when provider reads failed. Agents should explain uncertainty rather than inventing confidence.

### 17.2 Governance Risks

Governance can be captured, ignored, or misunderstood. Token voting is a mechanism, not wisdom. PaperProof governance should remain transparent about payloads, vote totals, locked funds, claim state, execution status, and authority boundaries.

### 17.3 Economic Non-Guarantees

This whitepaper does not promise token price, returns, liquidity, listing, revenue, grants, airdrops, user growth, or commercial success. Fees and incentives are protocol tools, not guarantees.

## 18 Conclusion

PaperProof begins with a simple belief: important digital works deserve durable protocol identities. Not every file needs this. Not every post should become protocol state. But some artifacts matter enough that their identity, versions, content commitments, interactions, governance records, and event history should outlive a single application.

Sui and Walrus make this practical. Sui provides object-centric state and cheap transactions. Walrus provides large content storage and static application surfaces [1, 2, 3]. PaperProof connects them into an artifact protocol with SDKs, indexers, frontends, and agent-readable workflows.

The official PaperProof application is only the first doorway. The larger goal is an ecosystem where researchers, developers, communities, data users, indexer operators, and agent builders can all treat PaperProof artifacts as durable public objects. If that ecosystem grows carefully, PaperProof can become a small but resilient layer for digital knowledge on Sui and Walrus.

## References


[1] Mysten Labs. *The Sui Smart Contracts Platform*. 2022. https://docs.sui.io/doc/sui.pdf

[2] Adam Welc and Sam Blackshear. "Sui Move: Modern Blockchain Programming with Objects." In *Companion Proceedings of the 2023 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, pages 53–55, 2023. doi: 10.1145/3618305.3623605.

[3] George Danezis, Giacomo Giuliari, Eleftherios Kokoris Kogias, Markus Legner, Jean-Pierre Smith, Alberto Sonnino, and Karl Wüst. "Walrus: An Efficient Decentralized Storage Network." arXiv:2505.05370, 2025. https://arxiv.org/abs/2505.05370

[4] Juan Benet. "IPFS - Content Addressed, Versioned, P2P File System." arXiv:1407.3561, 2014. https://arxiv.org/abs/1407.3561

[5] Protocol Labs. *Filecoin: A Decentralized Storage Network*. 2017. https://filecoin.io/filecoin.pdf

[6] Sam Williams, Viktor Diordiiev, Lev Berman, India Raybould, and Ivan Uemlianin. *Arweave: A Protocol for Economically Sustainable Information Permanence*. 2023. https://www.arweave.org/yellow-paper.pdf

[7] Lukas Weidener and Cord Spreckelsen. "Decentralized science (DeSci): definition, shared values, and guiding principles." *Frontiers in Blockchain*, 7:1375763, 2024. doi: 10.3389/fbloc.2024.1375763.

[8] Krzysztof Janowicz, Blake Regalia, Pascal Hitzler, Gengchen Mai, Stephanie Delbecque, Maarten Fröhlich, Patrick Martinent, and Trevor Lazarus. "On the prospects of blockchain and distributed ledger technologies for open science and academic publishing." *Semantic Web*, 9(5):545–555, 2018. doi: 10.3233/SW-180322.